Practica 2 TALF

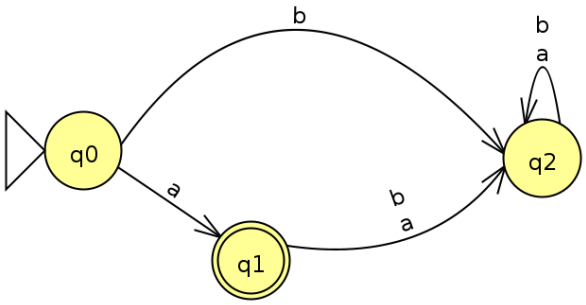
José Ruiz Pareja

29 de marzo de 2026

Considera el lenguaje sobre el alfabeto a, b que solamente contiene la cadena a: L = {a}.

1. Autómata finito en JFLAP:

Construye un AFD que reconozca el lenguaje y rechace todas las cadenas que no pertenezca al lenguaje.



Verifica el autómata construido con 6 cadenas de ejemplo.

Input	Result
	Reject
a	Accept
ab	Reject
aa	Reject
ba	Reject
aab	Reject

Observamos que el automata solo acepta la cadena a, y rechaza el resto: $\{\varepsilon, ab, aa, ba, aab\}$

2. Autómata finito en Python:

Abre el módulo automata y crea en Python un autómata equivalente al construido en la actividad anterior mediante finite_automaton.

Añadimos a *finiteautomata.json* el automata:

```
{
"name" : "solo_a",
"representation" : {
  "K" : ["q0", "q1", "q2"],
  "A" : ["a", "b"],
  "s" : "q0",
  "F" : ["q1"],
  "t" : [["q0", "b", "q2"],
        ["q0", "a", "q1"],
         ["q1", "b", "q2"],
         ["q1", "a", "q2"],
         ["q2", "a", "q2"],
         ["q2", "b", "q2"]]
  }
}
```

Ejecuta varias cadenas de prueba y comprueba cuáles son aceptadas.

```
>>> from python.automata import finite_automaton
>>> A = finite_automaton("solo_a", "ab")

M = ({q0, q1, q2}, {a, b}, {(q0, b, q2), (q0, a, q1), (q1, b, q2), (q1, a, q2), (q2, a, q2), (q2, b, q2)}, q0, {q1})

w = ab

(q0, ab) ⊢ (q1, b) ⊢ (q2, ε)

w ∉ ℒ(M)
>>> A = finite_automaton("solo_a", "a")

M = ({q0, q1, q2}, {a, b}, {(q0, b, q2), (q0, a, q1), (q1, b, q2), (q1, a, q2), (q2, a, q2), (q2, b, q2)}, q0, {q1})

w = a

(q0, a) ⊢ (q1, ε)

w ∈ ℒ(M)
>>> A = finite_automaton("solo_a", "b")

M = ({q0, q1, q2}, {a, b}, {(q0, b, q2), (q0, a, q1), (q1, b, q2), (q1, a, q2), (q2, a, q2), (q2, b, q2)}, q0, {q1})

w = b

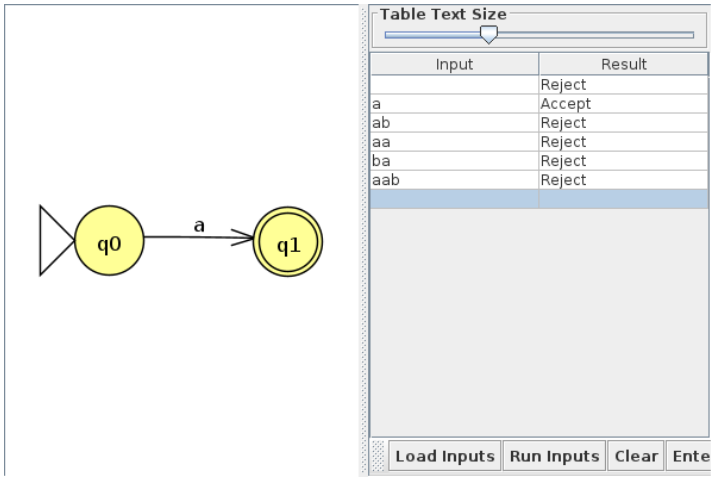
(q0, b) ⊢ (q2, ε)

w ∉ ℒ(M)
```

Como podemos ver solo acepta la cadena a y rechaza el resto de cadenas

3. Repite los pasos anteriores pero con un AFND y compara su comportamiento con el AFD.

Construye un AFND que reconozca el lenguaje y rechace todas las cadenas que no pertenezca al lenguaje. Verifica el autómata construido con 6 cadenas de ejemplo.



Crea en Python un autómata equivalente al construido en la actividad anterior mediante finite_automaton y ejecuta varias cadenas de prueba y comprueba cuáles son aceptadas.

```
{
    "name" : "solo_a_afnd",
    "representation" : {
      "K" : ["q0", "q1"],
      "A" : ["a", "b"],
      "s" : "q0",
      "F" : ["q1"],
      "t" : [["q0", "a", "q1"]]
      }
  }
```

```
>>> A = finite_automaton("solo_a_afnd", "a")

M = ({q0, q1}, {a, b}, {(q0, a, q1)}, q0, {q1})

w = a

(q0, a) ⊢ (q1, ε)

w ∈ ℒ(M)
>>> A = finite_automaton("solo_a_afnd", "b")

M = ({q0, q1}, {a, b}, {(q0, a, q1)}, q0, {q1})

w = b

(q0, b)
Blocked computation, w not accepted by M.
>>> A = finite_automaton("solo_a_afnd", "ab")

M = ({q0, q1}, {a, b}, {(q0, a, q1)}, q0, {q1})

w = ab

(q0, ab) ⊢ (q1, b)
Blocked computation, w not accepted by M.
>>> A = finite_automaton("solo_a_afnd", "aa")

M = ({q0, q1}, {a, b}, {(q0, a, q1)}, q0, {q1})

w = aa

(q0, aa) ⊢ (q1, a)
Blocked computation, w not accepted by M.
```